

The background features a dark blue gradient with a faint, repeating pattern of binary code (0s and 1s). Overlaid on this is a complex, glowing blue neural network diagram. The diagram consists of numerous nodes connected by lines, forming a dense web. A prominent dashed blue circle is centered in the middle of the image, framing the text. The overall aesthetic is high-tech and digital.

Redes Neurais Artificiais Algoritmo *Backpropagation*

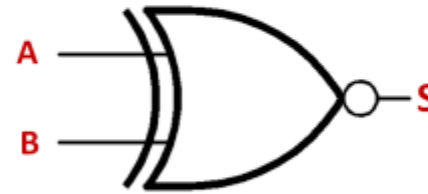
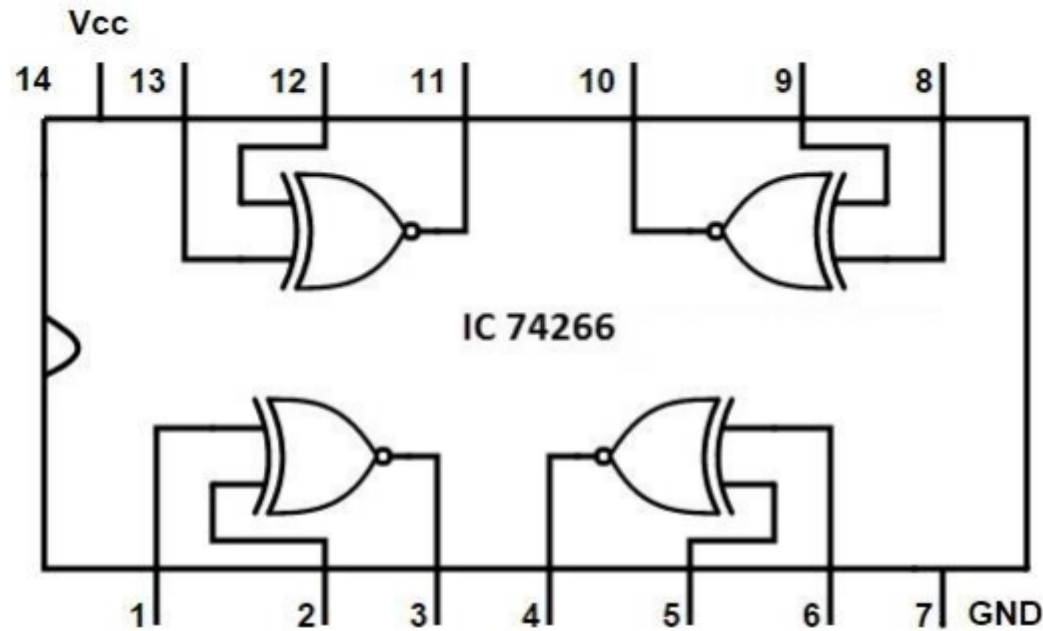
*Solução da função
booleana “E Coincidência”*

profº Mauricio Coneição Mario

Redes Perceptron – solução do problema E Coincidência

Saída terá nível lógico 1 se as entradas forem “iguais - coincidentes”.

Tabela da verdade da função Booleana “E Coinidência”

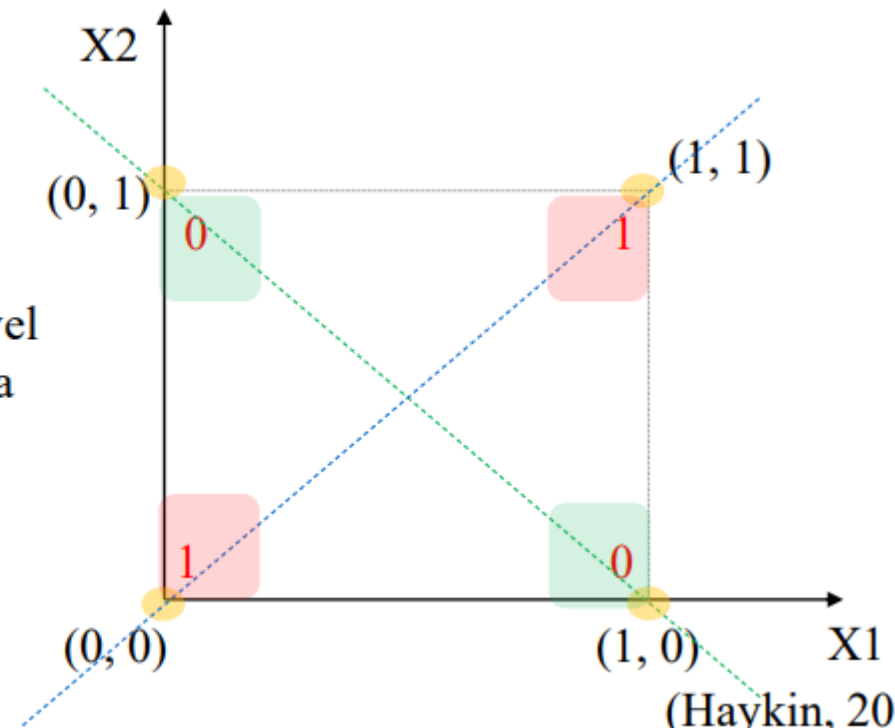


$$S = \bar{A}\bar{B} + AB$$

$$S = A \odot B$$

X1	X2	F
0	0	1
0	1	0
1	0	0
1	1	1

Função não é linearmente separável
(não se separam os espaços onde a
saída = 0 e saída = 1 por retas)

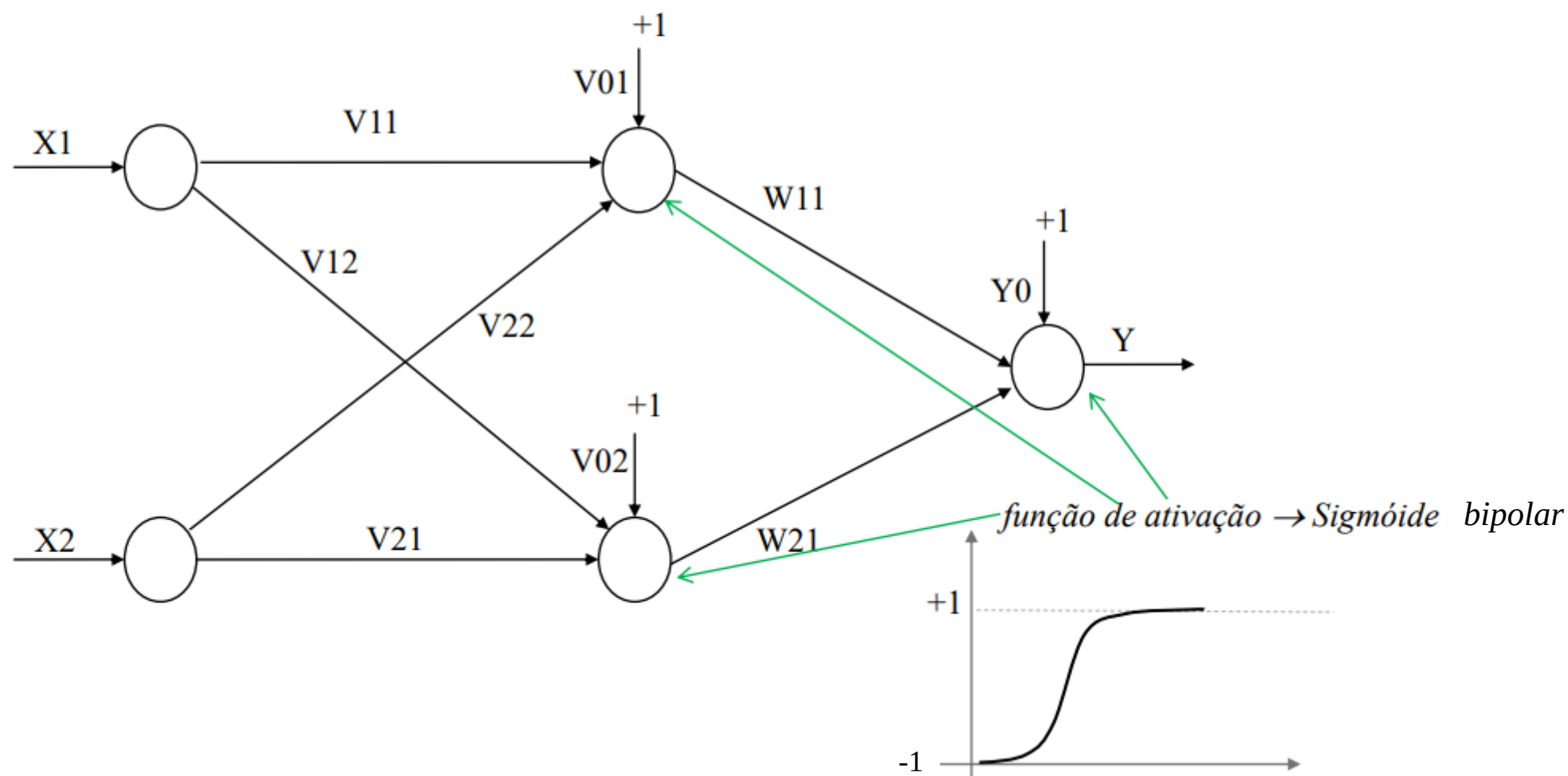


(Haykin, 2001)

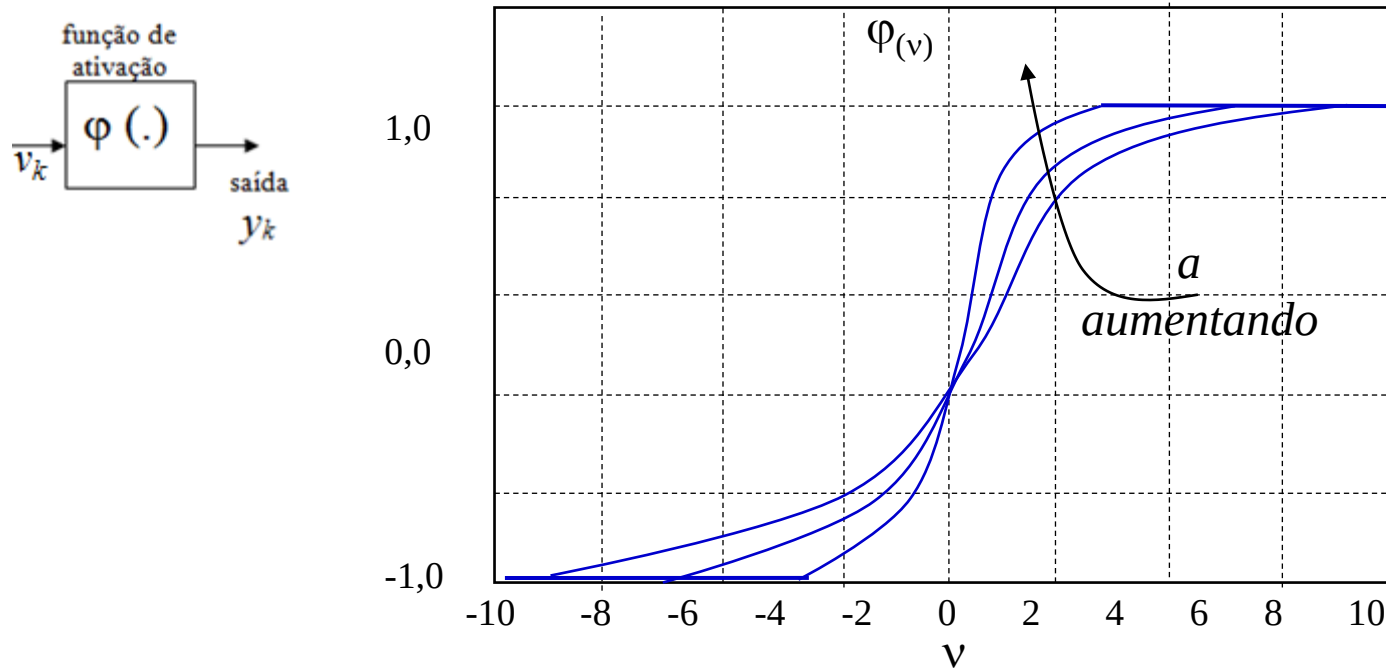
Solução da função *E Coincidência*

Implementação da função *E Coincidência* com algoritmo *Backpropagation*

***Rede Neural Artificial* com 1 camada oculta de 2 neurônios e 1 saída**



Redes Neurais Artificiais – funções de ativação



3. Função Sigmóide bipolar:

$$\varphi(v) = \frac{2}{1 + e^{-av}} - 1$$

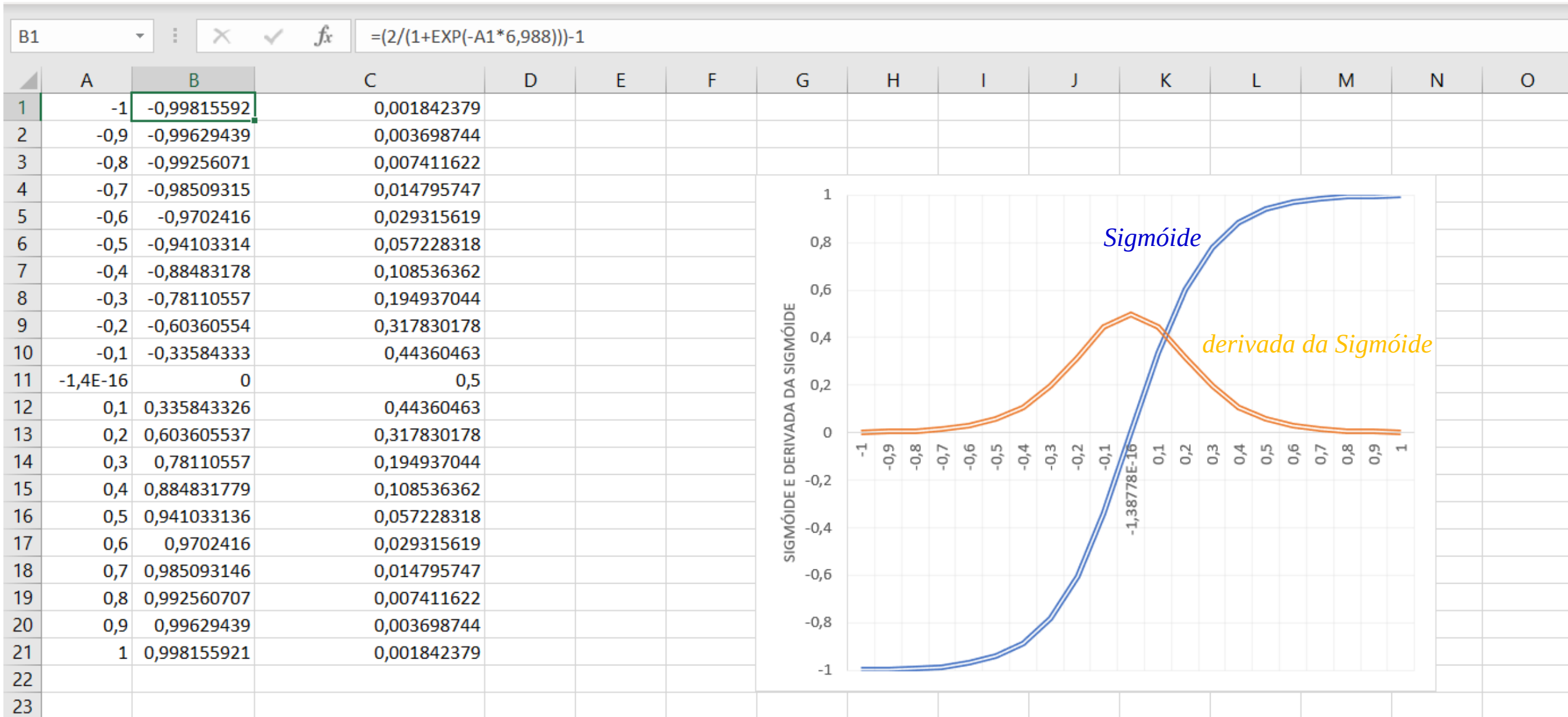
→ derivada

$$\varphi(v)' = 0.5 * (1 + \varphi(v)) * (1 - \varphi(v))$$

Onde a é o parâmetro de inclinação da função sigmóide.

(Haykin, 2001).

Sigmóide Bipolar e sua derivada



Algoritmo *Backpropagation*

1. Soma da junção aditiva nos neurônios da camada intermediária Z0 e Z1:

$$Z(x)_{[i]} = V0_{[i]} + v[i] * f(x)$$

2. Cálculo das funções de ativação sigmóide nos neurônios da camada intermediária:

$$Zf(x)_{[i]} = \frac{2}{1 + e^{-Z(x)[i]}} - 1$$

3. Cálculo da derivada da função de ativação:

$$Zff(x)_{[i]} = \frac{1}{2} * [(1 + Zf(x)[i]) * (1 - Zf(x)[i])]$$

4. Atualização de $Y(x)$:

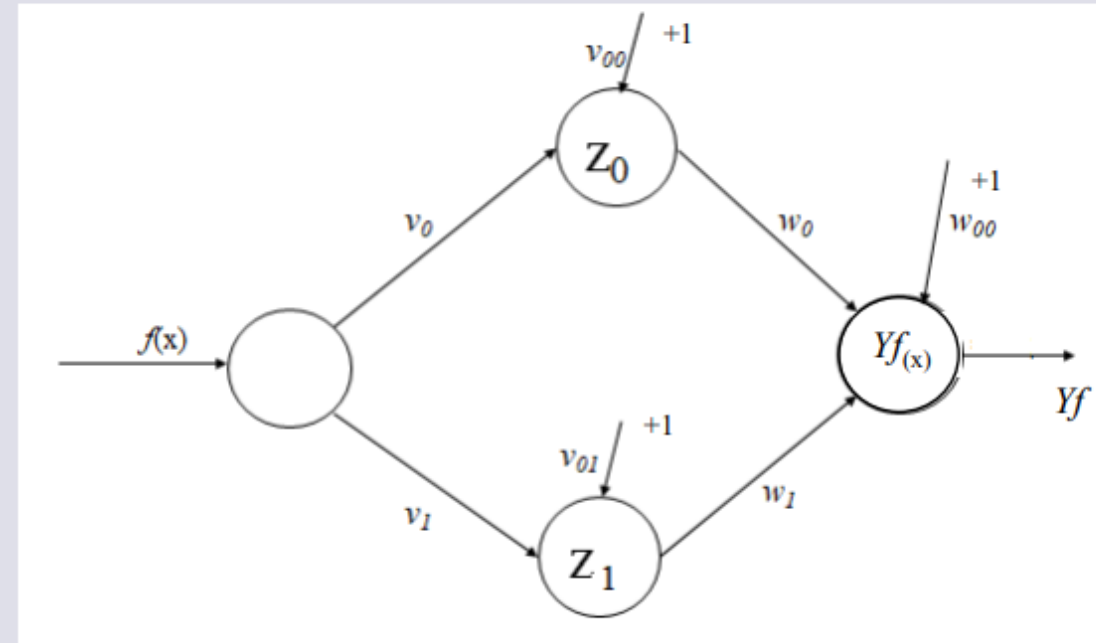
$$Y(x) = w_{00} + Zf[0] * w_0 + Zf[1] * w_1$$

5. Cálculo da função de ativação do neurônio na saída:

$$Yf[x] = \frac{2}{1 + e^{-Y(x)}} - 1$$

6. Cálculo do erro:

$$erro = (Y_D(x) - Yf(x))$$



7. Cálculo da derivada da função de ativação na saída:

$$Yff(x) = \frac{1}{2} * [(1 + Yf(x)) * (1 - Yf(x))]$$

Algoritmo Backpropagation

Retropropagação

8. Cálculo do erro atualizado: α = taxa de aprendizagem

$$\text{erro_atualizado}(x) = \alpha * \text{erro}(x) * Y_{ff}(x)$$

Atualização dos Δ e dos pesos das sinapses v e w

9. Atualização de Δw_{00} :

$$\Delta w_{00} = \alpha * \text{erro}(x)$$

10. Atualização de w_{00} :

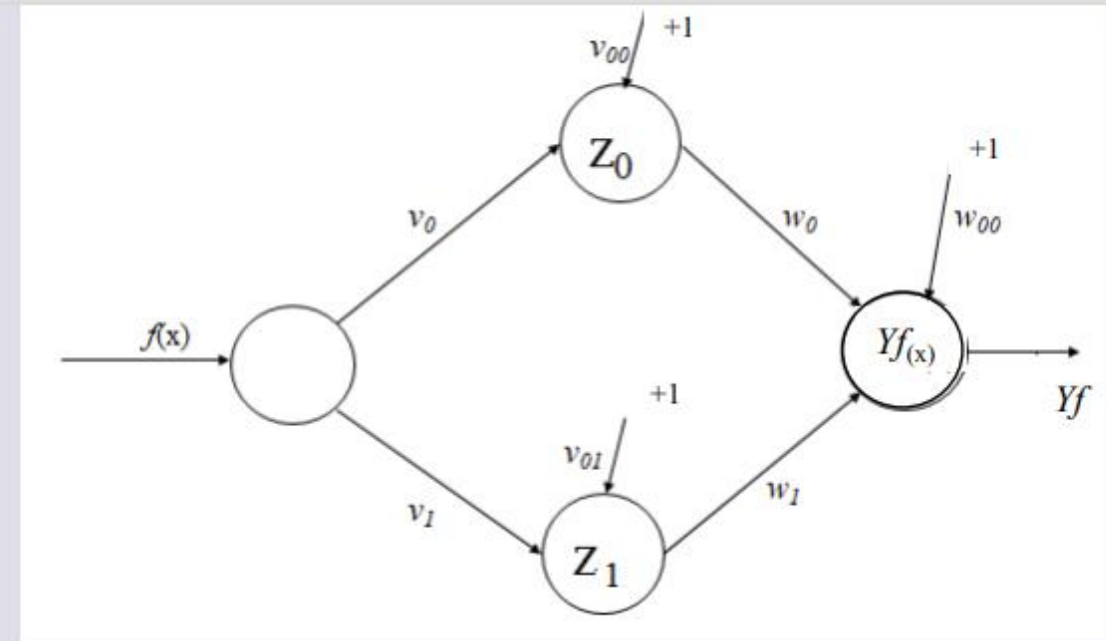
$$w_{00(n+1)} = w_{00(n)} + \Delta w_{00}$$

11. Atualização de $\Delta w_{[i]}$:

$$\Delta w_{[i]} = \text{erro_atualizado}(x) * Z_{ff}[i]$$

12. Atualização de $w_{[i]}$:

$$w_{[i](n+1)} = w_{[i](n)} + \Delta w_{[i]}$$



Atualização dos Δ e dos pesos das sinapses v e w

13. Atualização de $\Delta v_{0[i]}$:

$$\Delta v_{0[i]} = \alpha * \text{erro}(x) * Z_{ff}[i]$$

14. Atualização de $v_{0[i]}$:

$$v_{0[i](n+1)} = v_{0[i](n)} + \Delta v_{0[i]}$$

15. Atualização de $\Delta v_{[i]}$:

$$\Delta v_{[i]} = \Delta w_{[i]} * w_{[i]} * Z_{ff}[i] * f[x]$$

16. Atualização de $v_{[i]}$:

$$v_{[i](n+1)} = v_{[i](n)} + \Delta v_{[i]}$$

```
5 @author: cmari
6 """
7 from linear_algebra import dot
8 import math
9
10 def sigmoid(t):
11     return (2 / (1 + math.exp(-t)))-1
12
13 def neurônio_MCP(pesos, entradas):
14     uk = dot(pesos, entradas)
15     return sigmoid(uk)
16
17 def feed_forward(rede_xor, vetor_entrada):
18     """recebe uma rede neural (representada como uma lista de listas de listas de pesos)
19     e retorna a saída da propagação direta da entrada"""
20
21     vetor_saída = []
22
23     for ponteiro in rede_xor:
24
25         entrada_com_bias = vetor_entrada + [1] # adiciona bias ao vetor de entrada
26         saída = [neurônio_MCP(neurônio, entrada_com_bias) # calcula a saída
27                 for neurônio in ponteiro] # para a camada de rede
28         vetor_saída.append(saída) # adiciona ao vetor de saída
29
30         # a entrada para a próxima camada é a saída desta etapa
31         vetor_entrada = saída
32
33     return vetor_saída
34
35
```

→ função *backpropagation* recebe os pesos da rede, entrada e saída desejada.

→ invoca função *feed_forward* para produzir uma saída *sigmóide* em cada neurônio da Rede Neural: cada neurônio tem uma saída computada.

```
35
36 alpha = 0.08
37 def backpropagation(rede_neural, vetor_entrada, vetor_saída):
38     # feed_forward calcula a saída dos neurônios usando sigmóide
39     saídas_intermediárias, saídas_neurônios = feed_forward(rede_neural, vetor_entrada)
40     # 0.5 * alpha * (1 + output) * (1 - output) cálculo da derivada da sigmóide ""0.5*(output -
41     deltas_saída = [ (0.5 * (1 + saída) * (1 - saída)) * (saída - vetor_saída[i]) * alpha
42                     for i, saída in enumerate(saídas_neurônios)]
43
44     # ajuste dos pesos sinápticos para camada de saída (rede_neural[-1])
45     for i, neurônio_saída in enumerate(rede_neural[-1]):
46         for j, saída_intermediária in enumerate(saídas_intermediárias + [1]):
47             neurônio_saída[j] -= deltas_saída[i] * saída_intermediária
48
49     # 0.5 * alpha * (1 + output) * (1 - output) cálculo da derivada da sigmóide
50     # retro-propagação do erro para camadas intermediárias
51     deltas_intermediários = [ 0.5 * alpha * (1 + saída_intermediária) * (1 - saída_intermediária) *
52                               dot(deltas_saída, [n[i] for n in rede_neural[-1]])
53                               for i, saída_intermediária in enumerate(saídas_intermediárias)]
54
55     # ajuste dos pesos sinápticos para camadas intermediárias (network[0])
56     for i, neurônio_intermediário in enumerate(rede_neural[0]):
57         for j, input in enumerate(vetor_entrada + [1]):
58             neurônio_intermediário[j] -= deltas_intermediários[i] * input
59
```

Atualização de $\Delta w_{[i]}$:
 $\Delta w_{[i]} = \text{erro_atualizado}(x) * Zf[i]$

→ calcula o delta para a camada de saída, considerando a derivada da sigmóide, erro e taxa aprendizagem.

→ faz a atualização dos pesos sinápticos para a camada de saída

→ calcula o delta para a camada intermediária.

Atualização de $\Delta v_{[i]}$:
 $\Delta v_{[i]} = \Delta w_{[i]} * w_{[i]} * Zff[i] * f[x]$

→ faz a atualização dos pesos sinápticos para a camada intermediária.

Algoritmo Backpropagation

função *enumerate* desmembra os elementos de *saídas_intermediárias* para a variável *saída_intermediária*, correspondentes aos pesos sinápticos do segmento de saída da rede atualizados.

```

59
60
61 camada_intermediária = [[-0.1,-1.1, 1.55], [-0.91, -0.81, 0.125]]
62 camada_saída = [[0.2, -2.1, 0.98]]
63 rede_neural = [camada_intermediária, camada_saída]
64 print("camada intermediária = ", camada_intermediária)
65 print("camada saída = ", camada_saída)
66
67 entradas = [[0 ,0], [0, 1], [1, 0], [1, 1]]
68 saídas = [[1], [0], [0], [1]]
69
70 #TREINAMENTO DA REDE NEURAL
71 for i in range(30000):
72     for vetor_entrada, vetor_saída in zip(entradas, saídas):
73         backpropagation(rede_neural, vetor_entrada, vetor_saída)
74 #TREINAMENTO DA REDE NEURAL
75 print("pesos sinápticos atualizados \n", "camada intermediária = ", camada_intermediária, "\n", "camada saída = ", camada_saída)
76 #TESTE DA REDE NEURAL
77 for x in [0, 1]:
78     for y in [0, 1]:
79         print(x, "COINCIDÊNCIA", y, " = ", feed_forward(rede_neural, [x, y]) [-1])

```

Console 1/A

```

In [15]: runfile('D:/arquivos python/Backpropagation_E-Coincidência.py', wdir='D:/arquivos python')
camada intermediária = [[-0.1, -1.1, 1.55], [-0.91, -0.81, 0.125]]
camada saída = [[0.2, -2.1, 0.98]]
pesos sinápticos atualizados
camada intermediária = [[-1.6830140143111698, -1.736359976059576, 2.406704071675617], [-2.514424841124327, -2.624264170008664, 0.5106207448358295]]
camada saída = [[-7.0587679875989044, 7.5852594292176585, 8.234151099342265]]
0 COINCIDÊNCIA 0 = [0.9715370640739458]
0 COINCIDÊNCIA 1 = [0.0014112963834178593]
1 COINCIDÊNCIA 0 = [0.0014806992404972075]
1 COINCIDÊNCIA 1 = [0.9671622883157249]

```

Referências Bibliográficas

- *Python para Análise de Dados*
Tratamento de Dados com *Pandas*, *Numpy* e *Ipython*
Wes McKinney - O'REILLY
editora **novatec** – São Paulo – 2020
- *Data Science do Zero*
Primeiras Regras com o *Python*
Joel Grus - O'REILLY
editora **Alta Books** – Rio de Janeiro – 2016

Referências Bibliográficas

- Braga AP, Carvalho APLF, Ludermir TB. *Redes Neurais Artificiais: teoria e aplicações*. Livros Técnicos e Científicos, Rio de Janeiro – RJ; 2007.
- Haykin S. *Neural Networks – A Comprehensive Foundation*. Prentice-Hall; 1994.
- Haykin S. *Redes Neurais – Princípios e prática*. 2a ed.. Porto Alegre: Bookman; 2001.
- Hebb DO. *The Organization of Behavior*. John Wiley & Sons; 1949.
- Heckerman D. *Probabilistic Similarity Networks*. MIT Press, Cambridge, Massachussets; 1991.
- Hopfield JJ. *Neurons with graded response have collective computational properties like those of two-state neurons*. Proceedings of the National Academy of Sciences of the United States of America, 79, 2554-2558; 1982.
- Minsky, M.; Papert, P.. *Perceptrons: An introduction to computational geometry*. MIT Press, Massachussets, 1969.

Referências Bibliográficas

- Mario MC. *Proposta de Aplicação das Redes Neurais Artificiais Paraconsistentes como Classificador de Sinais Utilizando Aproximação Funcional*. Univ. Federal de Uberlândia, Dissertação de Mestrado, Uberlândia; 2003.
- McCarthy J. *Programs with common sense*. In Proceedings of the Symposium on Mechanisation of Thought Processes, Vol. 1, pp. 77-84, London. Her Majesty's Stationery Office; 1958.
- McCulloch W, Pitts W. *A logical calculus of the ideas immanent in nervous activity*. Bulletin of Mathematical Biophysics 5, 115-133; 1943.
- Rosenblatt F. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan, Chicago; 1962.
- Rumelhart DE, McClelland JL. *Parallel Distributed Processing*. MIT Press, Cambridge, Massachusetts; 1986.
- Turing A. *Computing machinery and intelligence*. Mind, 59, 433-460; 1950.

Referências Bibliográficas

- Medeiros, L.F.. *Redes Neurais em Delphi*. Editora Visual Books. Florianópolis, 2003.
- Cybenko, G.. *Continuous valued neural networks with two hidden layers are sufficient*, Technical report, Department of Computer Science, Tufts University, 1988.
- Cybenko, G.. *Approximation by superpositions of sigmoid function*, *Mathematics of Control, Signals and Systems*, 2:303-314, 1989.